



12. Movimientos para personajes y NPCs

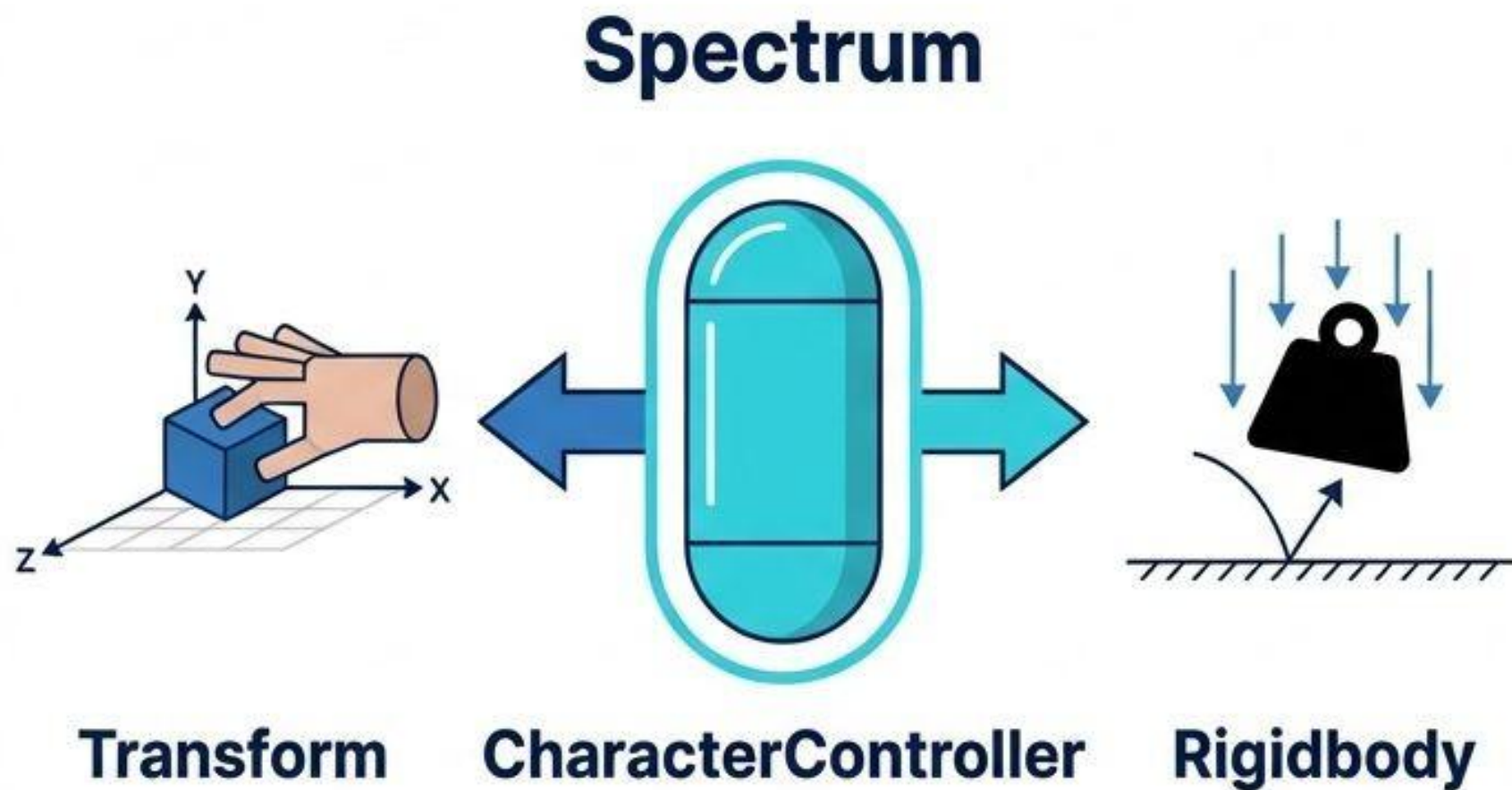
Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

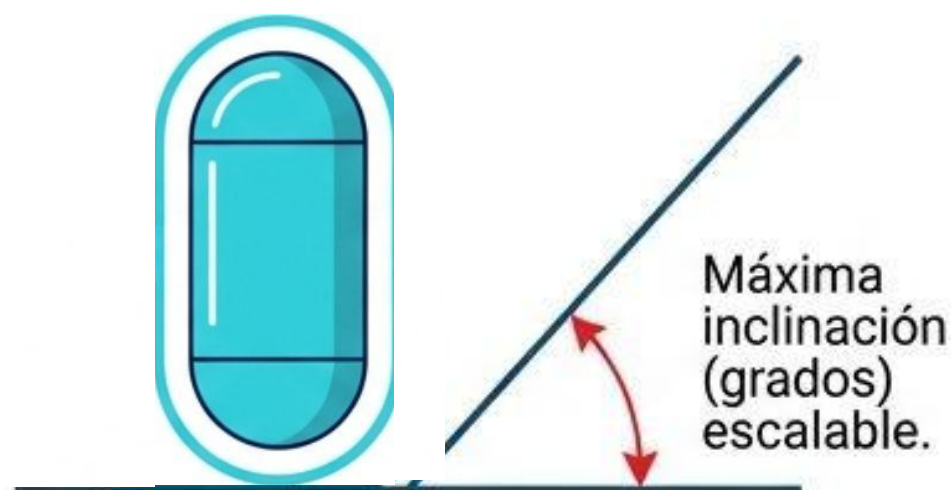
Fernando Madrid Recio

Definición: El Híbrido Cinemático



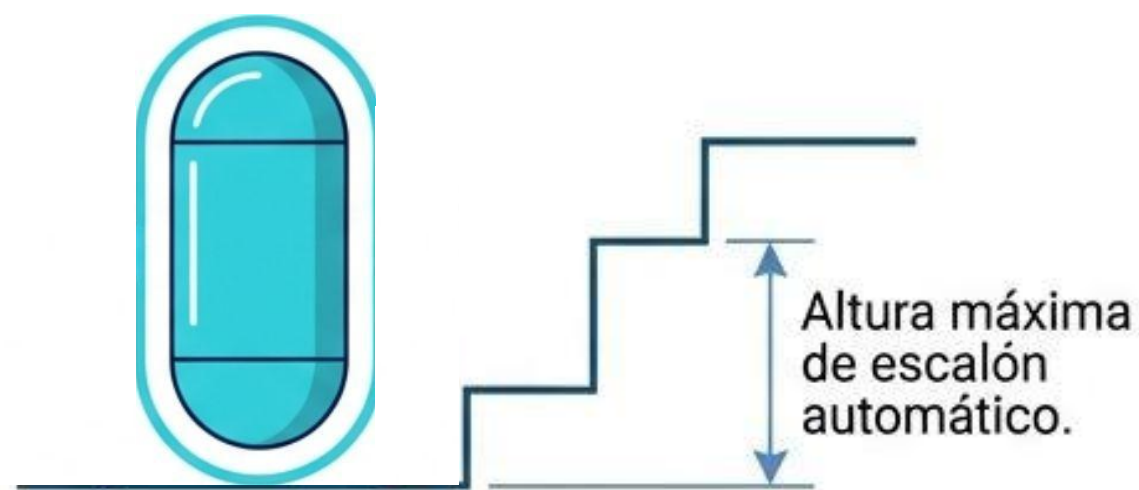
- **Híbrido:** Combina el control absoluto del Transform con la detección de entorno del Rigidbody.
- **Sin Física Real:** No tiene inercia, masa ni responde a fuerzas externas automáticamente.
- **Restricciones:** No atraviesa paredes ni suelos (Collision Awareness).
- **Uso Principal:** Personajes humanoides (FPS/TPS) que requieren movimiento preciso.

Configuraciones Clave del Componente



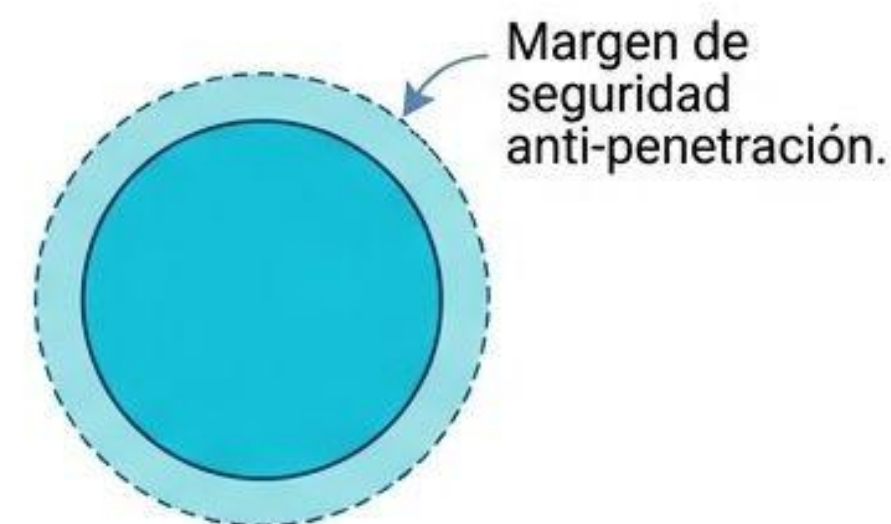
Slope Limit

Máxima inclinación (grados) escalable.



Step Offset

Altura máxima de escalón automático.



Skin Width

Margen de seguridad anti-penetración.

El Método Move()



```
// Referencia al componente
CharacterController controller = GetComponent<CharacterController>();

// Cálculo del movimiento (Input + Gravedad)
Vector3 velocity = moveDirection * speed;
velocity.y -= gravity * Time.deltaTime; // Gravedad manual

// Ejecutar movimiento
controller.Move(velocity * Time.deltaTime);
```

La Regla de Oro de Move()

Precaución: Una sola llamada por Frame



INCORRECTO

```
controller.Move(walkVector);  
controller.Move(gravityVector);
```

Dos llamadas provocan **errores**
de colisión y jitter.



CORRECTO

```
Vector3 finalVector = walkVector  
+ gravityVector;  
controller.Move(finalVector);
```

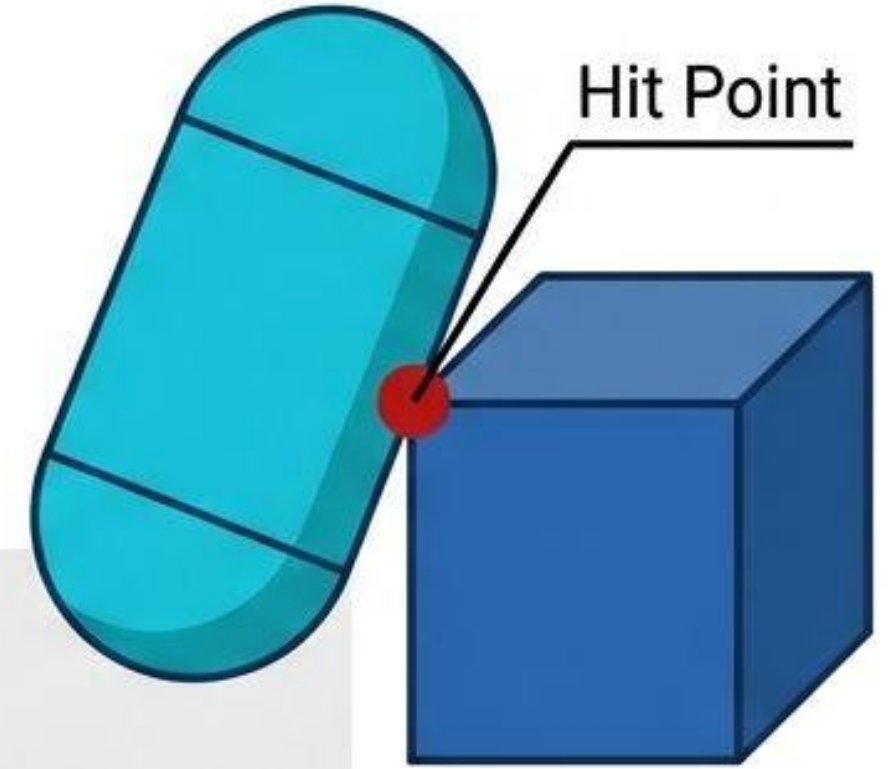
Sumar **vectores** primero, mover
una sola vez.

Detección de Colisiones e Interacción

OnCollisionEnter no es fiable para el
CharacterController.
Usamos **OnControllerColliderHit**.

```
void OnControllerColliderHit(ControllerColliderHit hit)
{
    Rigidbody body = hit.collider.attachedRigidbody;

    // Solo empujar si el objeto tiene física y no es cinemático
    if (body != null && !body.isKinematic) {
        body.linearVelocity = hit.moveDirection * pushPower;
    }
}
```



Conclusiones: ¿Cuál elegir?

Característica	CharacterController	Rigidbody
Control	Preciso / Instantáneo (FPS)	Simulado / Con Inercia
Gravedad	Cálculo Manual (Código)	Automática (Built-in)
Colisiones	No empuja por defecto	Reacción física real
Escaleras	Sube automáticamente (Step Offset)	Requiere física compleja / Rampas

Usa **CharacterController** para el jugador si buscas control preciso.
 Usa **Rigidbody** para objetos físicos, vehículos y ragdolls.

NavMeshSurface: El Escenario

Concepto:

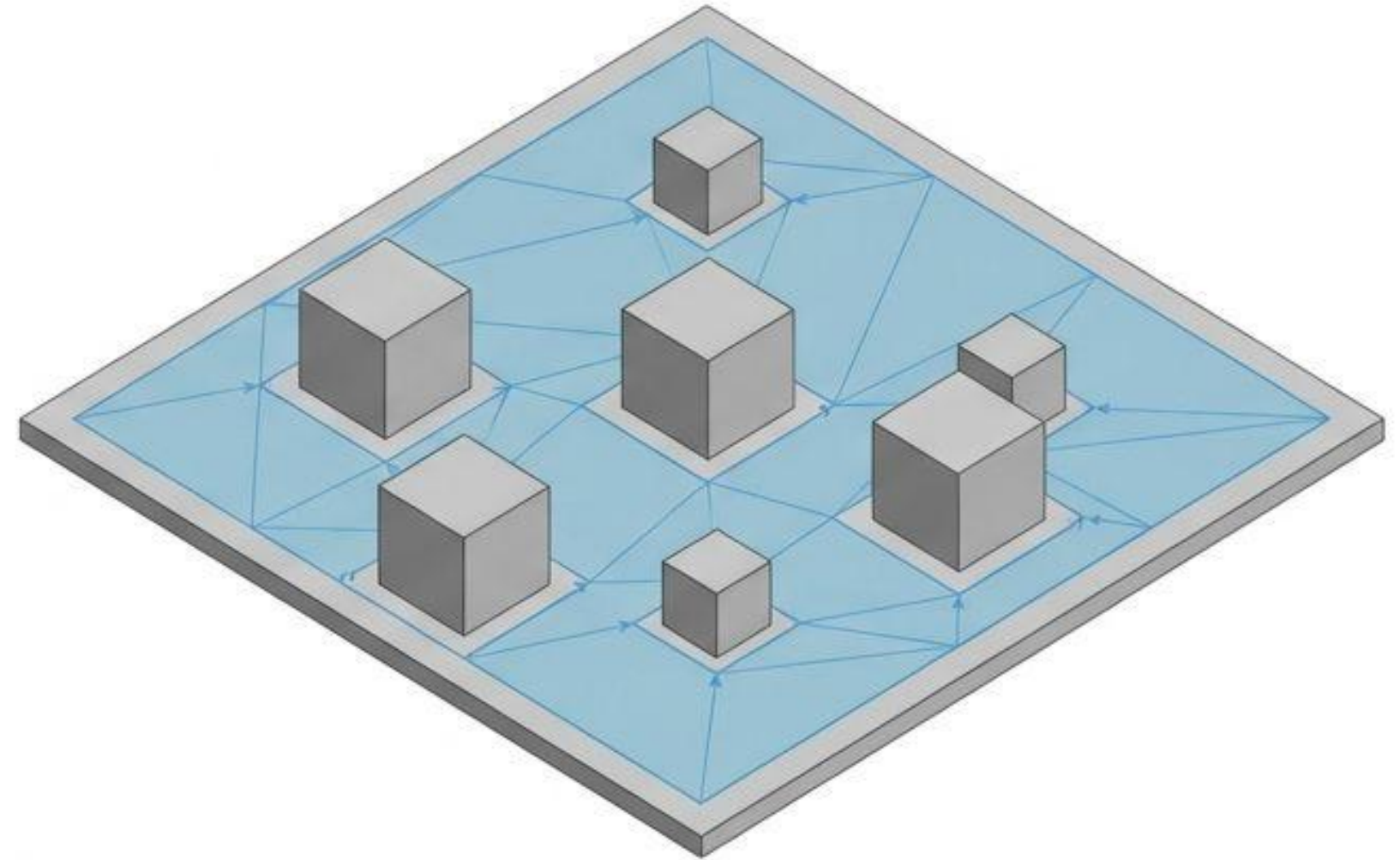
El proceso de 'Bake' analiza la geometría estática del nivel.

Función:

Determina qué áreas son transitables y cuáles son obstáculos.

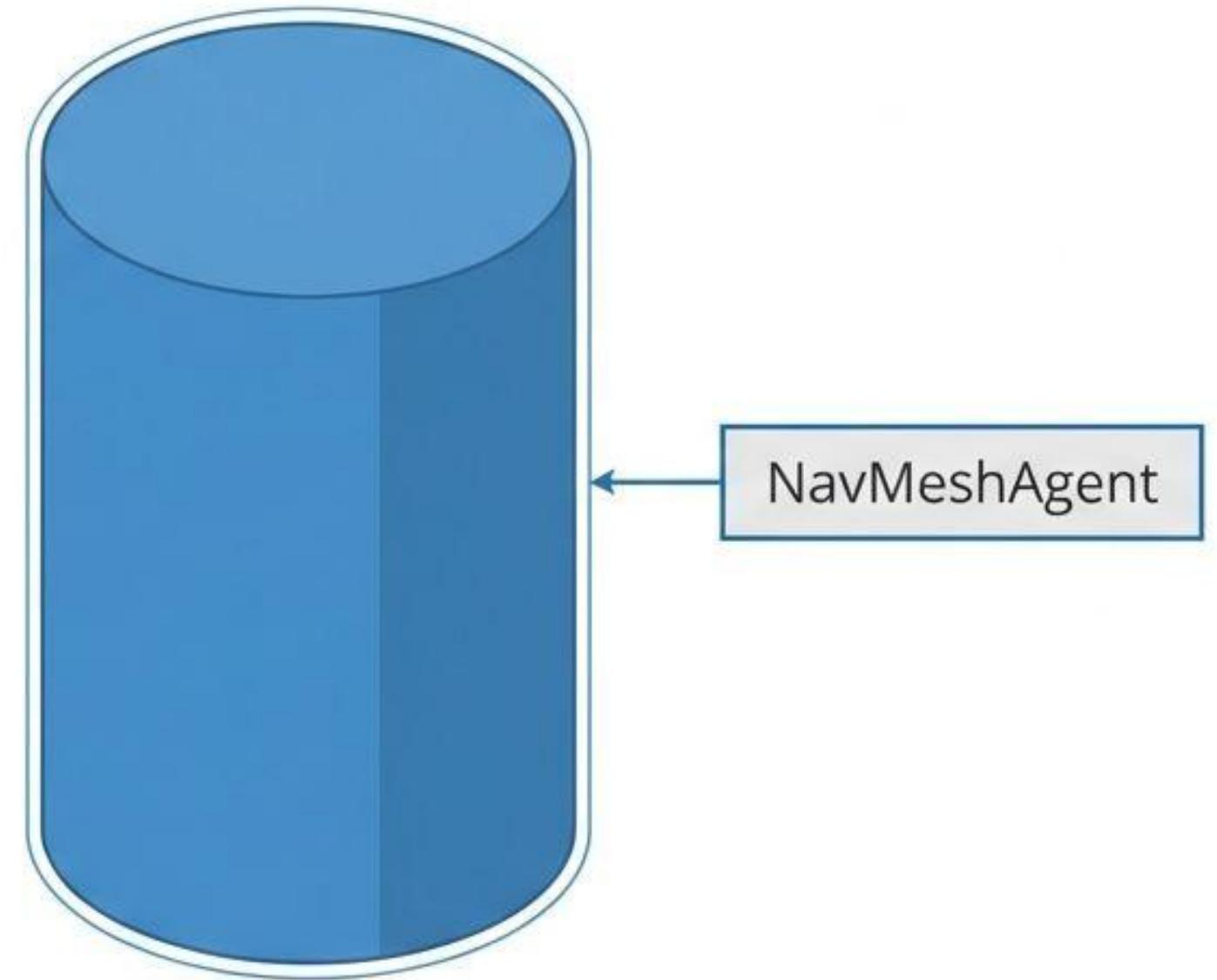
Resultado:

Genera una malla de navegación invisible sobre el terreno (Walkable Areas).



El Actor: NavMeshAgent

- **Definición**
 - Componente esencial adjunto a enemigos o NPCs.
- **Responsabilidad**
 - Calcula rutas usando el algoritmo A* (A-Star).
- **Movimiento**
 - Desplaza al personaje evitando obstáculos dinámicos y estáticos automáticamente.



Configuraciones del NavMeshAgent



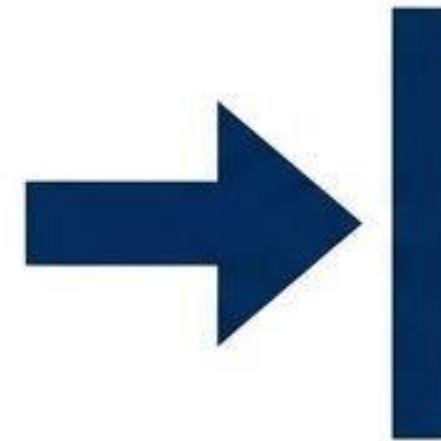
Speed: Velocidad máxima de movimiento lineal.



Angular Speed: Velocidad de rotación (grados/segundo).



Acceleration: Qué tan rápido alcanza la velocidad máxima.



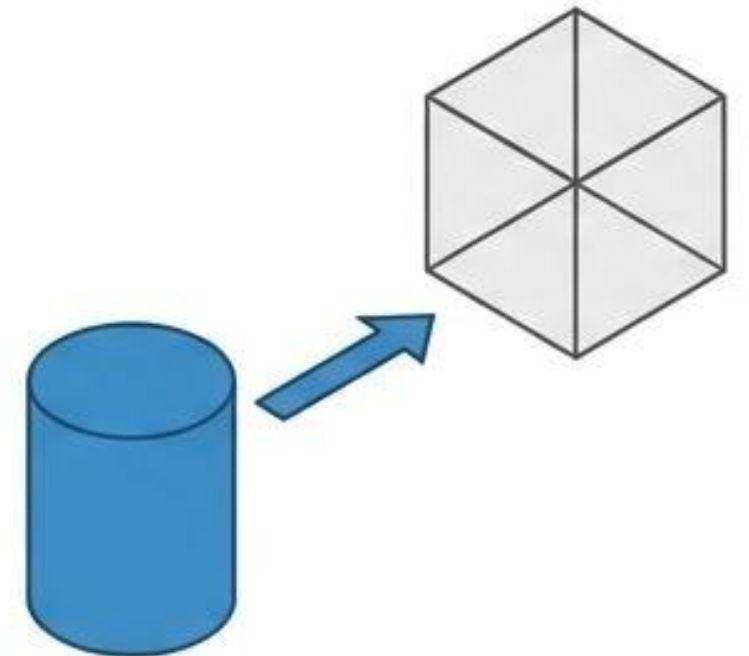
Stopping Distance: Distancia a la que se detiene antes de tocar el objetivo.

Asignar Objetivos: SetDestination()

- Calcula la ruta más corta de forma asíncrona.
- No requiere control frame a frame (como en Update).

```
public Transform target;
```

```
// Asigna el destino al NavMeshAgent  
agent.SetDestination(target.position);
```



Verificación de Llegada

Es necesario comprobar dos condiciones:

1. Si la ruta ha terminado de calcularse (!pathPending).
2. Si la distancia restante es menor a la distancia de frenado.

```
// Comprobación robusta de llegada
if (!agent.pathPending &&
    agent.remainingDistance <= agent.stoppingDistance)
{
    // Lógica de ataque o siguiente patrulla
    Debug.Log('La IA ha llegado al destino');
}
```

